MINI PROJECT REPORT

SMART PROCUREMENT SYSTEM

A C and MySQL Based Inventory, Supplier and Quotation Management Application

(BATCH-2 DIV-13 CSF PANEL-A)

Group Members -	Mihir Gondse -	1262252478
	Prathemesh Patil -	1262252789
	Pranav Patil -	1262252853
	Neha Avhad -	1262252511

1. Abstract

The Smart Procurement System is a menu-driven, terminal-based application developed in C and connected to a MySQL database. It is designed to help manage products, stock quantities, suppliers and supplier quotations from one place. The program supports adding, viewing, searching, updating and deleting product records; adding and viewing supplier records; recording and viewing quotations; sorting products by unit price; generating a low-stock report; calculating total inventory value; and displaying summary information on a dashboard.

The application uses the MySQL C API to send SQL queries to a database named **smart_procurement**. The database stores persistent records in related tables, while C structures represent product, supplier and quotation data during input. The system is suitable as a learning mini project because it demonstrates C structures, functions, loops, switch-case menus, SQL queries, database connectivity and basic reporting.

Keywords: C programming, MySQL, procurement, inventory, supplier management, quotation, database connectivity.

2. Introduction

Procurement is the process of obtaining goods or services from suppliers. A basic procurement workflow needs accurate product records, stock quantities, supplier contact details and quotations so that prices can be compared before purchasing. When these records are kept manually, it can become difficult to find information quickly or notice low stock.

The Smart Procurement System provides a simple computer-based solution. The user selects an option from the terminal menu and the program executes the corresponding C function. SQL statements read or change information in MySQL. This separation allows the program logic to be written in C while the database handles persistent storage.

2.1 Objectives

- Maintain product details and current stock.
- Store supplier names, contact numbers and email addresses.
- Record quotations against suppliers and products.
- Search for products and suppliers by name.
- Identify products with quantity below 10 units.
- Calculate total inventory value using quantity × unit price.
- Compare recorded quotations by price and display the lowest-priced quotation.

2.2 Scope

The current version is a local, terminal-based application for a single database connection. It does not implement user login, purchase orders, quotation approval, automatic stock updates from purchases, or a graphical/web interface. These can be considered future improvements.

3. ER Diagram

The following Chen-style diagram represents the main entities and relationships implied by the program's SQL queries. Entities are shown as rectangles, relationships as diamonds and attributes as ellipses; primary keys are underlined. A supplier can be associated with multiple products and can submit multiple quotations. A product can appear in multiple quotations. Each quotation refers to one supplier and one product.

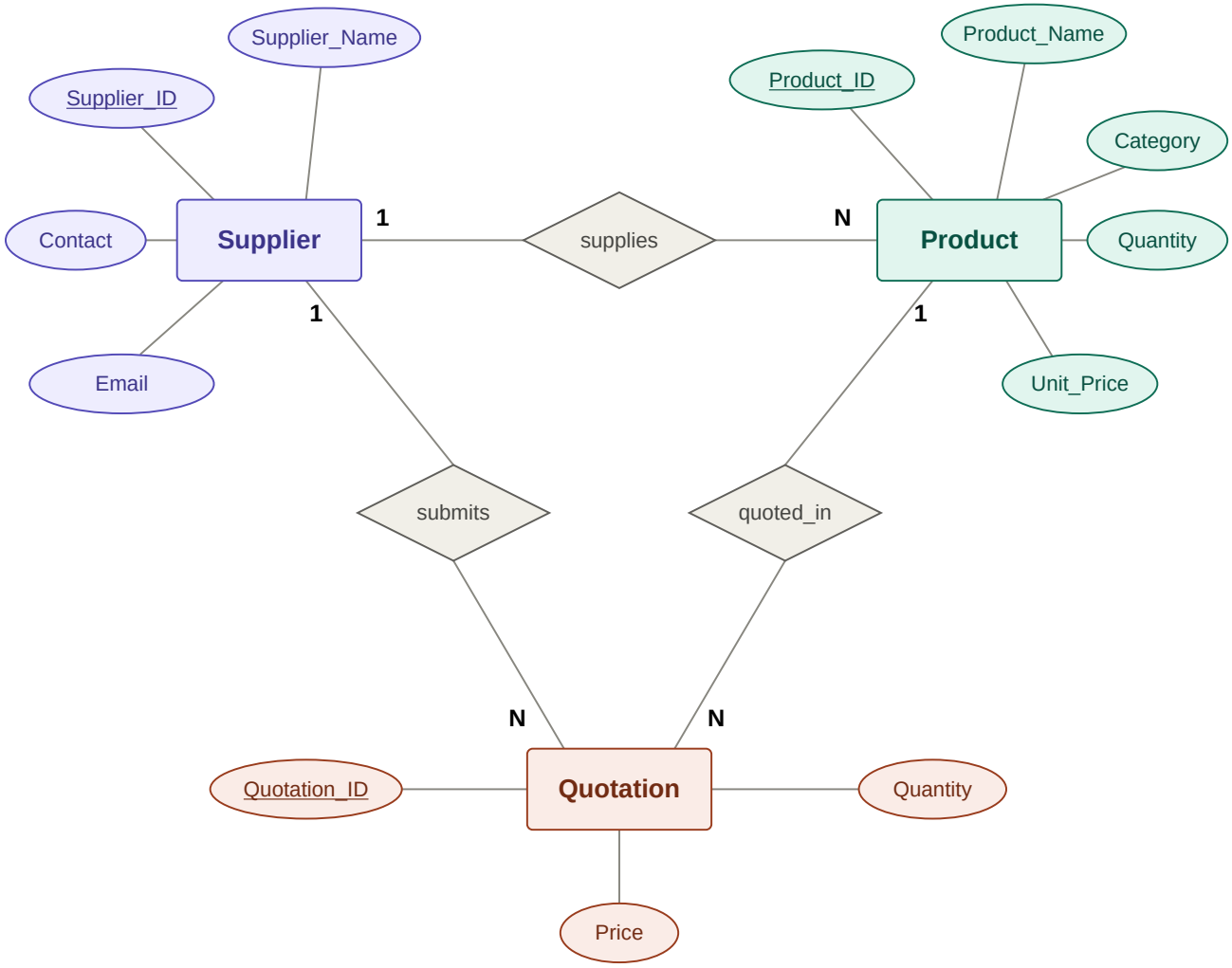

Figure 1. Chen ER diagram for the three main tables.

Relationship notes

- **supplies (1 : N)** — one Supplier supplies many Products. Products.Supplier_ID is intended to reference Suppliers.Supplier_ID.
- **submits (1 : N)** — one Supplier submits many Quotations. Quotations.Supplier_ID is intended to reference Suppliers.Supplier_ID.
- **quoted_in (1 : N)** — one Product appears in many Quotations. Quotations.Product_ID is intended to reference Products.Product_ID.
- Suppliers.Supplier_ID, Products.Product_ID and Quotations.Quotation_ID are the primary keys of their tables.

Important: The C program checks that a supplier exists before adding a product, and checks that the supplier and product exist before adding a quotation. For full database-level integrity, define matching foreign-key constraints in the MySQL schema as well.

4. Table Schema

The code expects a MySQL database named **smart_procurement** and uses three tables: Products, Suppliers and Quotations. The exact table definitions were not included in the supplied C source, so the following schema is a recommended schema compatible with the column names used by the program.

```
CREATE DATABASE IF NOT EXISTS smart_procurement;
USE smart_procurement;

CREATE TABLE Suppliers (
    Supplier_ID   INT PRIMARY KEY,
    Supplier_Name VARCHAR(50) NOT NULL,
    Contact       VARCHAR(20),
    Email         VARCHAR(80)
);

CREATE TABLE Products (
    Product_ID    INT PRIMARY KEY,
    Product_Name  VARCHAR(50) NOT NULL,
    Category      VARCHAR(50),
    Quantity      INT NOT NULL DEFAULT 0,
    Unit_Price    DECIMAL(10,2) NOT NULL,
    Supplier_ID   INT NOT NULL,
    FOREIGN KEY (Supplier_ID)
        REFERENCES Suppliers(Supplier_ID)
);

CREATE TABLE Quotations (
    Quotation_ID  INT PRIMARY KEY,
    Supplier_ID   INT NOT NULL,
    Product_ID    INT NOT NULL,
    Quantity      INT NOT NULL,
    Price         DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (Supplier_ID)
        REFERENCES Suppliers(Supplier_ID),
    FOREIGN KEY (Product_ID)
        REFERENCES Products(Product_ID)
);
```

4.1 Table description

Table	Purpose	Important columns
Suppliers	Stores supplier contact information.	Supplier_ID (PK), Supplier_Name, Contact, Email
Products	Stores inventory/product records.	Product_ID (PK), Product_Name, Category, Quantity, Unit_Price, Supplier_ID (FK)
Quotations	Stores quoted prices and quantities.	Quotation_ID (PK), Supplier_ID (FK), Product_ID (FK), Quantity, Price

PK = Primary Key; FK = Foreign Key. DECIMAL is used for prices to avoid common floating-point storage issues in database currency fields.

5. Application

The application runs in a terminal. At startup, the program initializes a MySQL connection to localhost and selects the smart_procurement database. If the connection fails, the program prints the MySQL error and exits. If the connection succeeds, it displays the main menu in a loop until the user selects Exit.

5.1 Main menu features

Menu option	Functionality
1. Dashboard	Displays counts of products, suppliers and quotations, total stock units, and inventory value.
2. Add Product	Accepts product details and checks for duplicate product IDs and an existing supplier.
3. View Products	Lists all products with category, quantity, unit price and supplier ID.
4. Search Product	Searches product names using SQL LIKE matching.
5. Update Stock	Replaces a product's quantity after checking the product exists and quantity is not negative.
6. Delete Product	Deletes a product by ID, subject to database constraints.
7. Add Supplier	Adds supplier details after checking for duplicate supplier IDs.
8. View Suppliers	Displays stored supplier details.
9. Search Supplier	Searches suppliers by name.
10. Add Quotation	Stores a quotation for an existing supplier and product.
11. View Quotations	Joins quotations with suppliers and products to show readable names.
12. Lowest Quotation	Displays the quotation row with the lowest recorded Price.
13. Sort Products by Price	Lists products in ascending order of Unit_Price using ORDER BY.
14. Low Stock Report	Lists products where Quantity is below 10, lowest quantity first.
15. Inventory Value	Calculates SUM(Quantity × Unit_Price).
0. Exit	Returns from the menu and closes the database connection in main().

5.2 Technologies used

C language: Handles menu control, input, validation, functions and output.

MySQL: Stores product, supplier and quotation records persistently.

MySQL C API: Connects the C application to MySQL and executes SQL commands.

SQL: Uses SELECT, INSERT, UPDATE, DELETE, JOIN, ORDER BY, COUNT and SUM operations.

ANSI terminal escape sequences: Add colors and clear the terminal on compatible consoles.

5.3 Basic workflow

1. Start the program.
2. Connect to the MySQL database.
3. Select an option from the menu.
4. Enter the requested data or search criteria.
5. The C function runs the relevant SQL query.
6. The application displays a success message, result, or database error.
7. Press Enter to return to the menu, or select 0 to exit.

5.4 Notes and limitations in the supplied code

- The database password is written directly in the source code. For a real deployment, store credentials outside source code and use a restricted database account. Do not publish the password.
- SQL queries that insert or search text are built with sprintf(). Names containing apostrophes can break these queries, and untrusted input can create SQL-injection risks. Prepared statements should be used instead.
- Input reads such as scanf(" %[^\n]") do not specify a maximum field width, so long input can overflow fixed-size arrays. Safer bounded input should be used.
- The lowest quotation function sorts by Price only. It does not compare total cost, delivery time, quality or quotation quantity; therefore it reports the lowest listed unit/quoted price, not necessarily the best overall supplier.
- Deleting a product may fail if a quotation references it and foreign-key restrictions are enabled.
- Terminal colors and CLEAR may not behave the same in every console.

6. Future Improvements

The current program covers the core data-management tasks for a small procurement workflow. The following improvements could make it safer and more useful:

1. **Prepared SQL statements:** Replace sprintf-built queries with parameterized statements to handle special characters safely and reduce SQL-injection risk.

2. **Input validation:** Validate numeric input, email format, non-negative prices and quantities, and maximum text lengths.

3. **Credential security:** Move database credentials to a configuration file or environment variable and use a dedicated database account with limited privileges.

4. **Login and roles:** Add authentication and separate administrator, procurement and read-only access.

5. **Purchase-order workflow:** Add purchase orders, approval status, order dates, delivery status and payment tracking.

6. **Better quotation comparison:** Compare quotations for the same product and required quantity, and consider taxes, delivery charges, lead time and supplier reliability.

7. **Automatic stock movement:** Update stock through receiving and issuing transactions, with an audit history.

8. **Reports and export:** Export reports to CSV or PDF and add date filters.

9. **Graphical or web interface:** Create a GUI or web frontend while keeping business logic and database access in a backend service.

10. **Testing and backups:** Add test cases, transaction handling, logging, and scheduled database backups.

7. Conclusion

The Smart Procurement System demonstrates how a C application can connect to MySQL and perform useful procurement and inventory operations. It uses structures to organize input data, functions to separate features, a switch-case menu for navigation, and SQL queries for storing, retrieving and analyzing records. Its reporting features include low-stock detection, inventory valuation, price sorting and lowest-quotation lookup.

The project is suitable for demonstrating foundational programming and database concepts. Before using it for real business data, it should be improved with prepared statements, safer input handling, protected credentials, stronger validation and database constraints.

8. References

1. MySQL Documentation — MySQL C API: https://dev.mysql.com/doc/c-api/8.4/en/

2. MySQL Documentation — SQL Statements: https://dev.mysql.com/doc/refman/8.4/en/sql-statements.html

3. ISO C library reference — stdio.h, stdlib.h and string handling concepts.

4. The supplied Smart Procurement System C source code, used as the basis for this report.